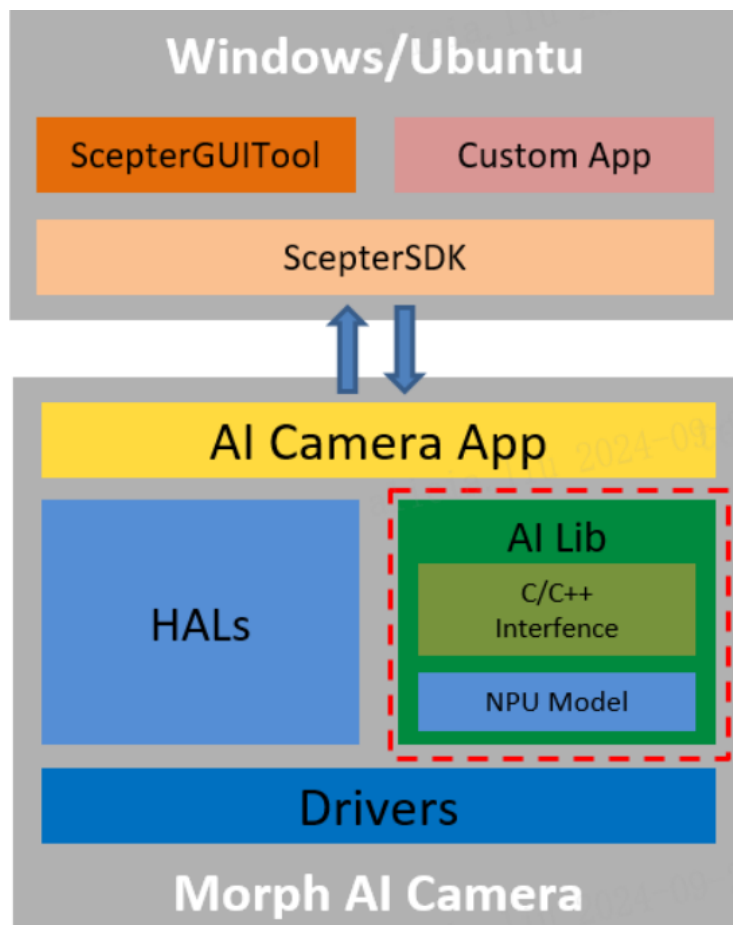


ScepterSoftware&MorphAI相机编程指南

1 简介

ScepterSoftware由ScepterSDK与ScepterGUITool组成。ScepterSDK 是基于 3D ToF 相机提供的软件开发包，与普通相机相比，增加了AI-Lib模块相关API，方便用户快速开发验证自己的机器视觉算法。ScepterGUITool 是基于 Scepter SDK 开发的图形界面工具，除了标准相机的功能设置外，AI Module使用后，还可以进行AI算法相关设置及算法效果预览，方便开发者开发调试自己的AI算法，快速实现AI算法的边缘部署。



ScepterSDK 下载

ScepterSDK 下载链接：

<https://github.com/ScepterSW/ScepterSDK>

您可以通过以下方式下载相应开发包：

打开下载链接，点击 Code，复制链接，git clone获取develop分支。

```
1 | $ git clone -b develop --single-branch
   | https://github.com/ScepterSW/ScepterSDK.git
```

ScepterGUITool 下载

ScepterGUITool 下载链接：

ScepterGUITool有两种安装方式：安装版与绿色版。根据个人使用习惯选择对应操作系统的版本即可。

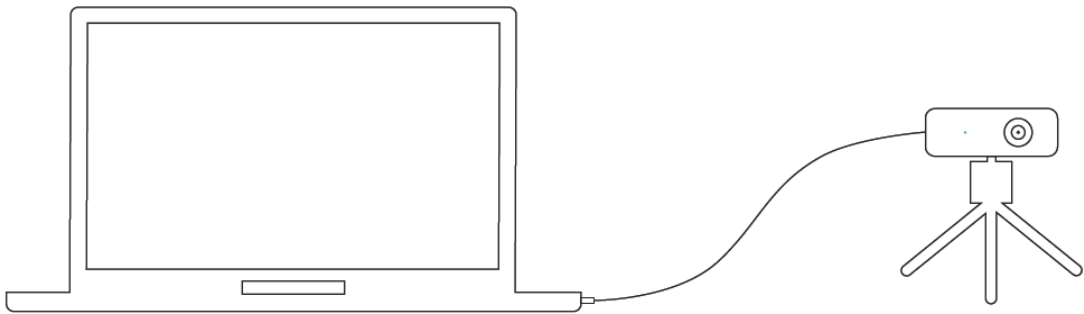
2 连接

2.1 推荐系统配置

配置项	Windows推荐配置	Ubuntu推荐配置
操作系统	Windows10 64 位 Windows11 64 位	Ubuntu 18.04 64 位 Ubuntu 20.04 64 位 Ubuntu 22.04 64 位
内存	4g 以上	4g 以上

2.2 设备连接

将相机安装在一个合适的固定装置中，例如相机支架。



- 1、通过以太网线/航插网线将相机连接至主机；
- 2、将电源线连接至相机，等待相机指示灯闪烁，完成设备连接。

网线连接分为固定 IP 地址直连与 DHCP 连接两种方式，设备默认使用固定 IP 地址方式连接，如需更改 IP 地址、子网掩码、DHCP，可以使用 ScepterGUITool 进行更改。

PC 端使用的网卡、路由器、交换机都要满足千兆要求。

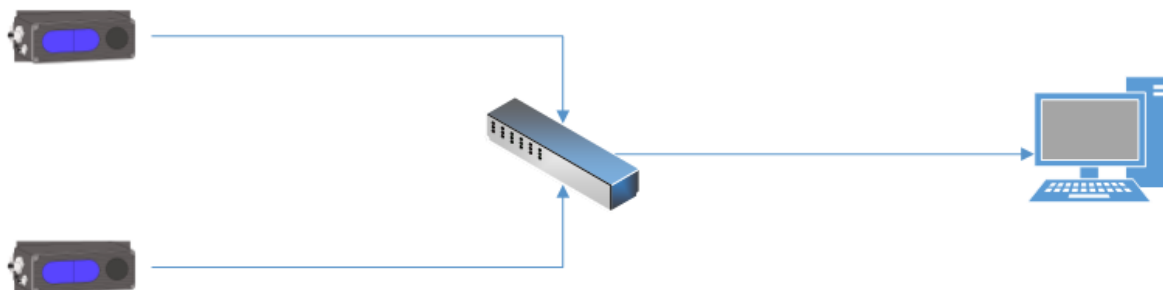
2.2.1 固定地址

固定地址连接可以设备与电脑直连，也可以配置在同一网段的交换机中使用。

直连：一端连接设备，另一端连接 PC 主机的网线接口。设备默认 IP 为 192.168.1.101，在 PC 端将“本地连接”的，子网掩码设为 255.255.255.0，IP 地址设为同一网段（如 192.168.1.100）。



多个相机连接：需要注意，因为相机默认 IP 地址都为 192.168.1.101 连接同一个 PC 主机是会出现 IP 冲突，所以要更改相机的默认 IP 地址（如相机 1 的 IP 地址：192.168.1.101，相机 2 的 IP 地址：192.168.1.102），可参考[vzense wiki文档中3.5. 设备网络设置](#)更改相机 IP。



主机 IP 地址需要与相机在同一网段，需要更改主机的IP地址。

Windows平台更改IP地址步骤如下：

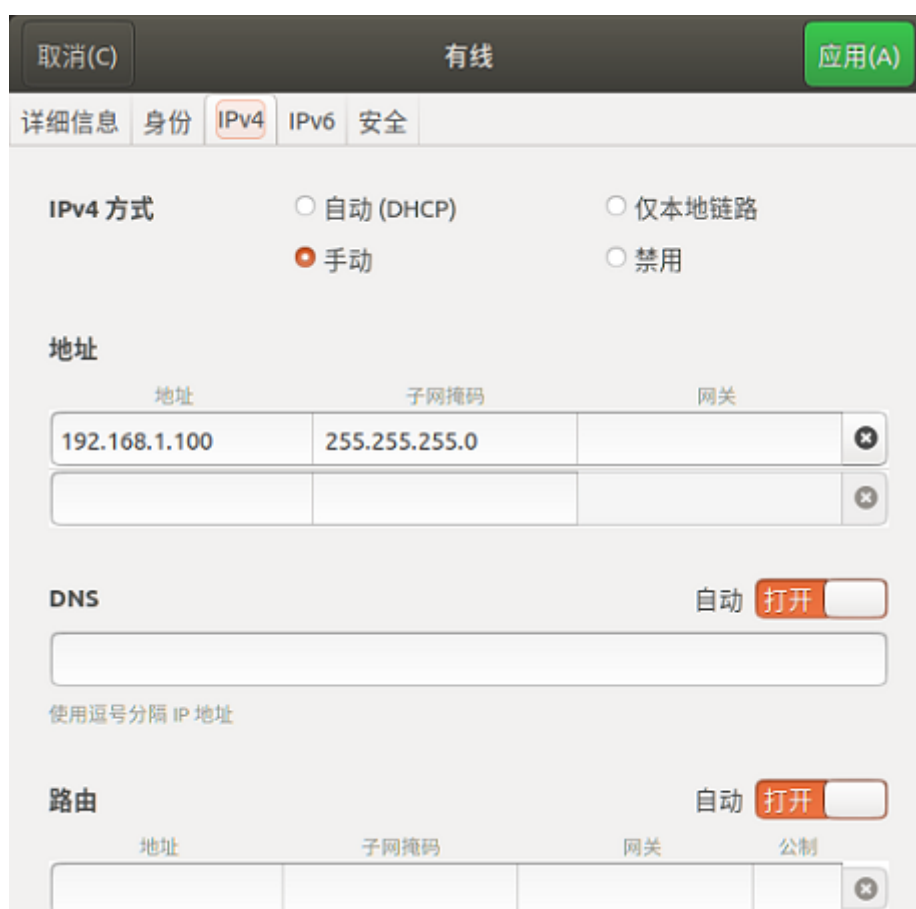
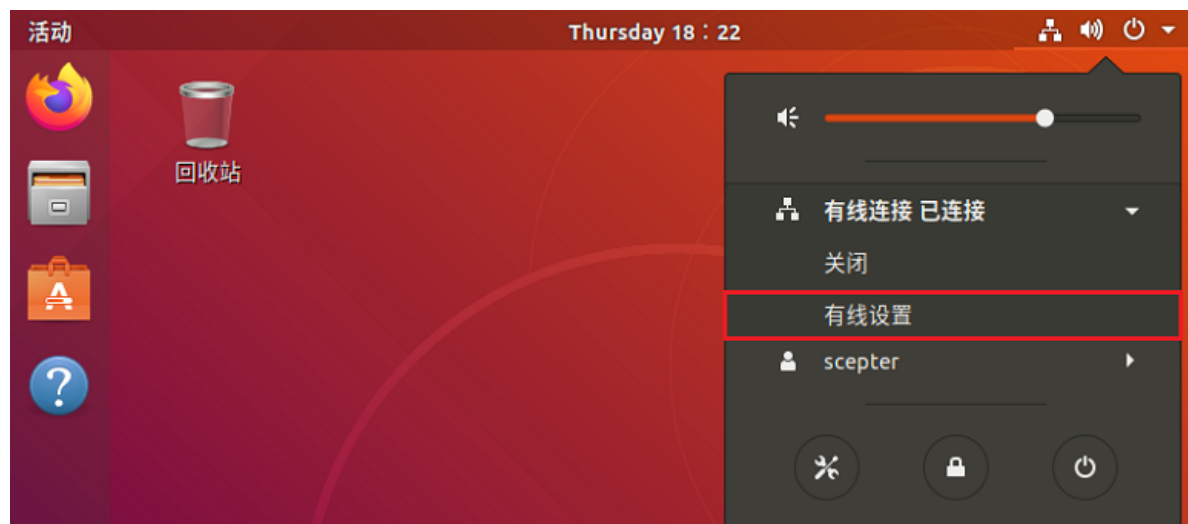
The image shows a sequence of three Windows operating system windows demonstrating how to change the IP address of a network adapter.

1. **网络和共享中心 (Network and Sharing Center):** The window shows the network status for the 'MOS.COM' network. The '更改适配器设置' (Change adapter settings) link is highlighted with a red box.

2. **网络连接 (Network Connections):** This window displays a list of network adapters. The '以太网 2' (Ethernet 2) adapter is selected, and the '属性(R)' (Properties) button is highlighted with a red box.

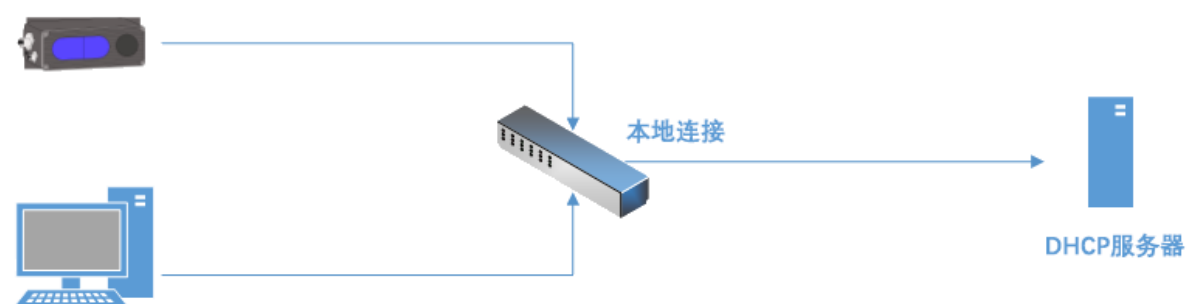
3. **以太网 属性 (Ethernet Properties):** This window shows the configuration for the selected Ethernet adapter. The 'Internet 协议版本 4 (TCP/IPv4) 属性' (Internet Protocol Version 4 Properties) tab is active. Under the '常规' (General) section, the '使用下面的 IP 地址(S):' (Use the following IP address) option is selected. The IP address is set to 192.168.1.100, the subnet mask is 255.255.255.0, and the default gateway is left blank. The '高级(V)...' (Advanced...) button is visible at the bottom right.

Ubuntu平台更改IP地址步骤如下：



2.2.2. DHCP

DHCP 连接方式，需要将相机连接在开启 DHCP 功能的路由器上，使用在相同局域网中的 PC 进行连接。设置相机 DHCP 的方法，请参考普通相机 ScepterGUITool 的文档。推荐将 PC 的“本地连接”设置为自动获取 IP 地址。



Windows平台设置DHCP的步骤如下：



Ubuntu平台设置DHCP的步骤如下：

取消(C)

有线

应用(A)

详细信息

身份

IPv4

IPv6

安全

IPv4 方式

☒ 自动 (DHCP)

☐ 仅本地链路

☐ 手动

☐ 禁用

DNS

自动

打开

使用逗号分隔 IP 地址

路由

自动

打开

地址

子网掩码

网关

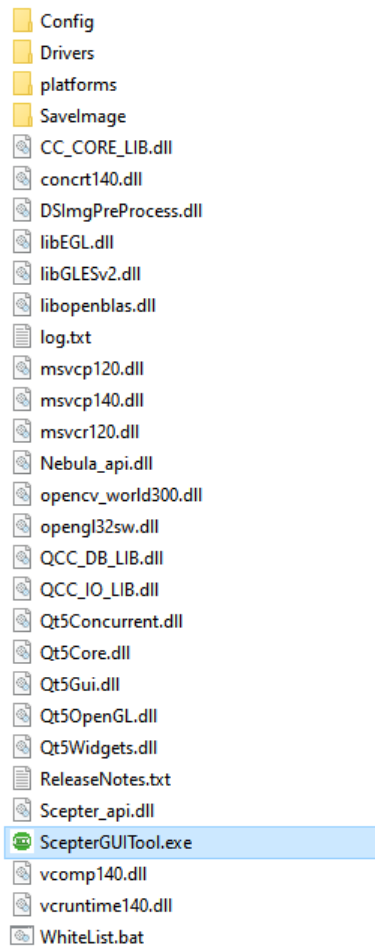
公制

×

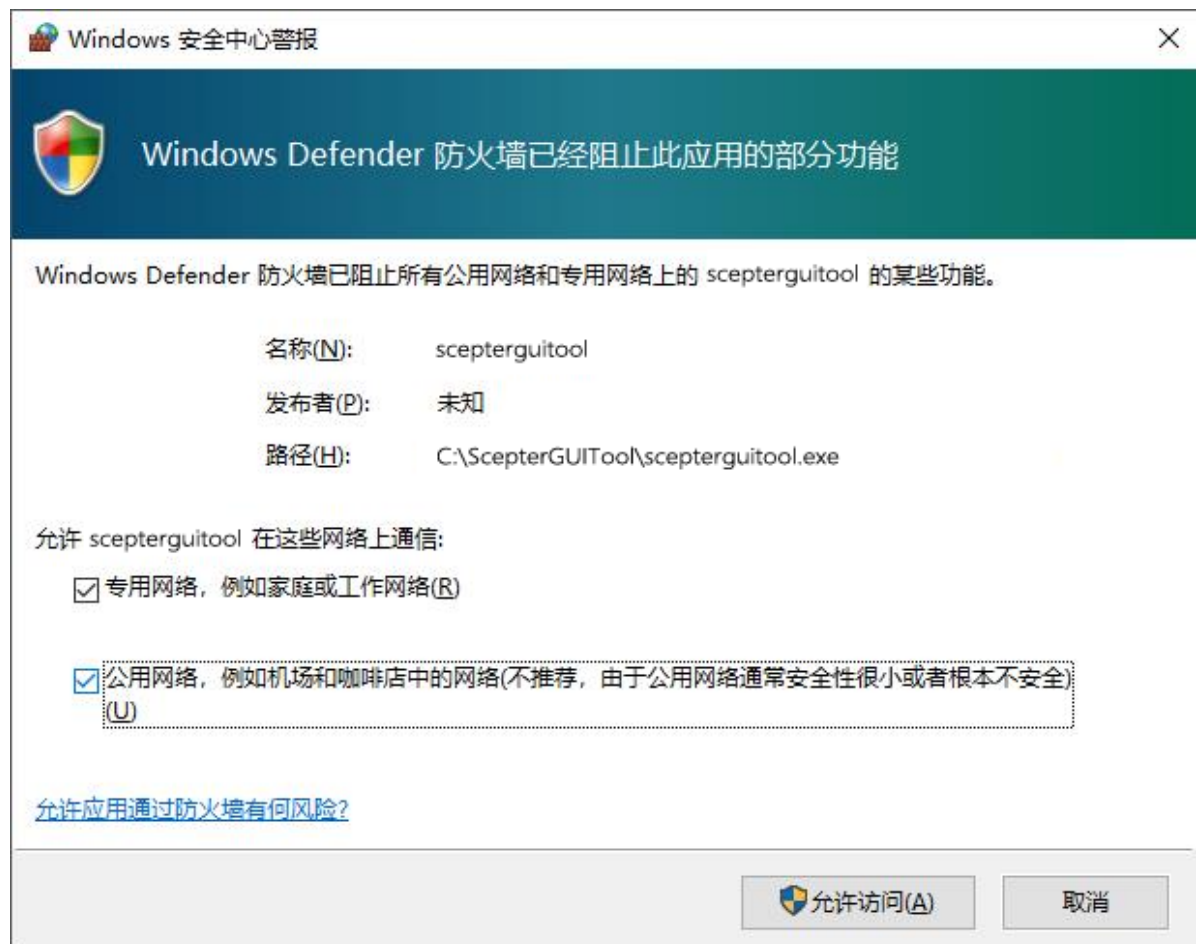
☐ 仅对该网络上的资源使用此连接(O)

2.3 打开设备

运行下载安装的ScepterGUITool。

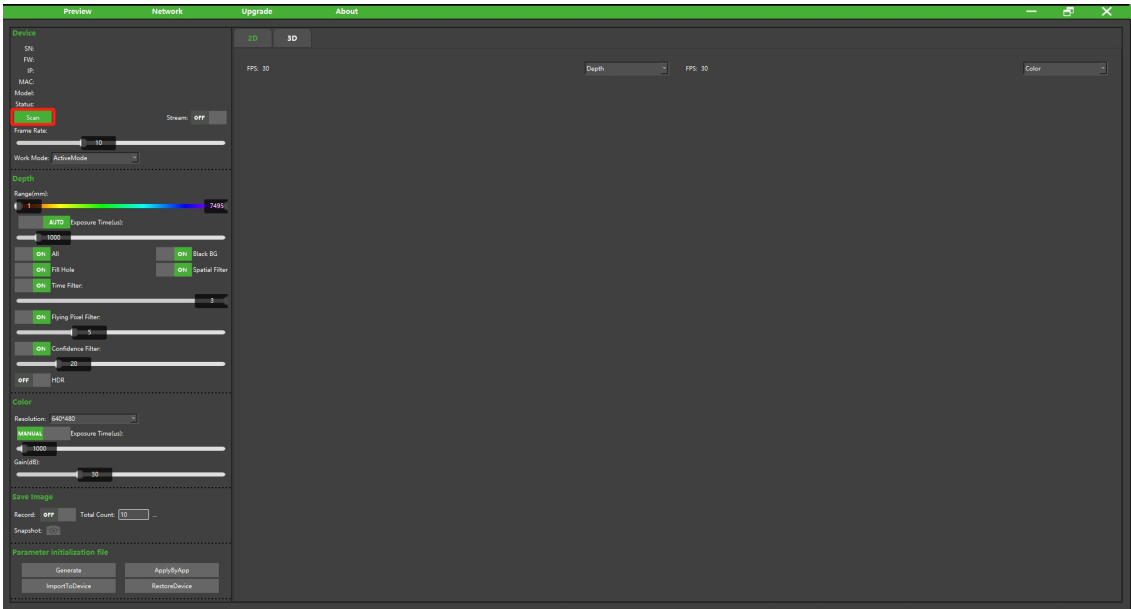


在首次运行 ScepterGUITool 时，要为程序设置通过系统防火墙的权限，如下图所示：

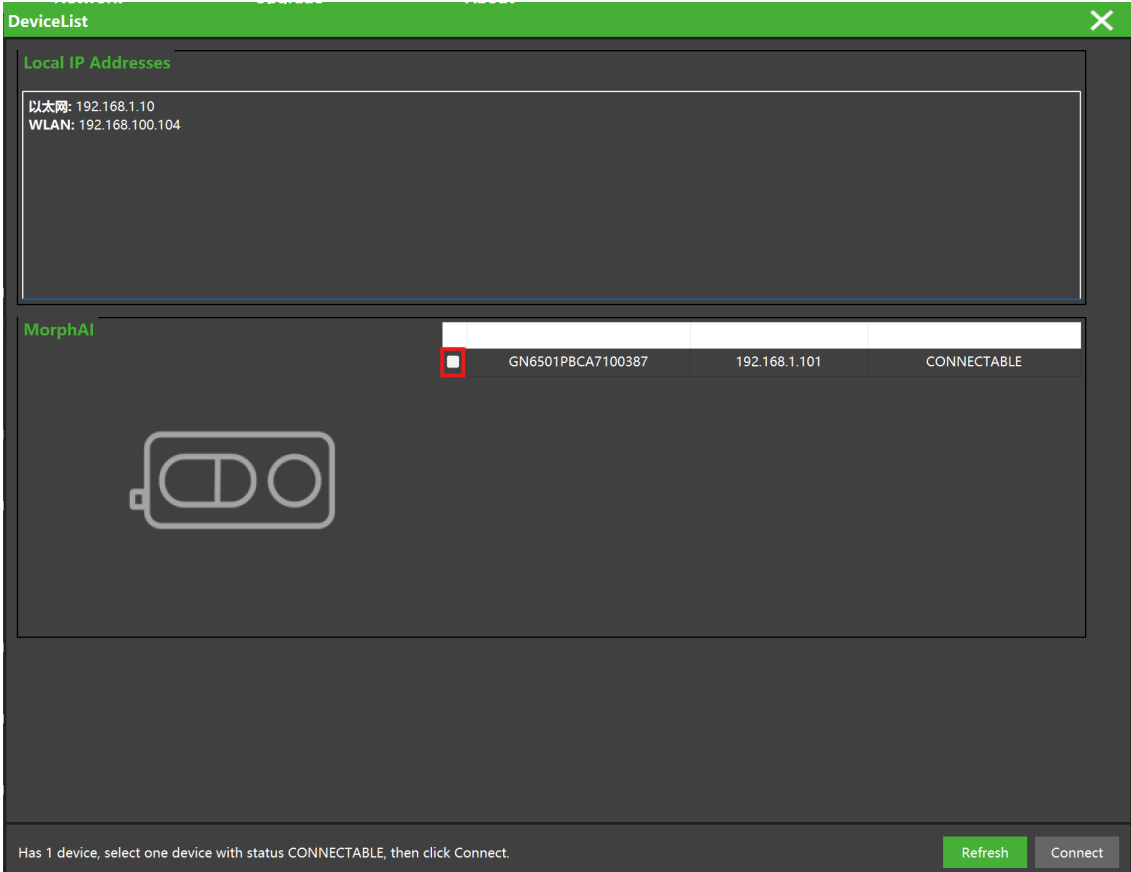


双击 ScepterGUITool 可执行文件，按照以下步骤连接设备：

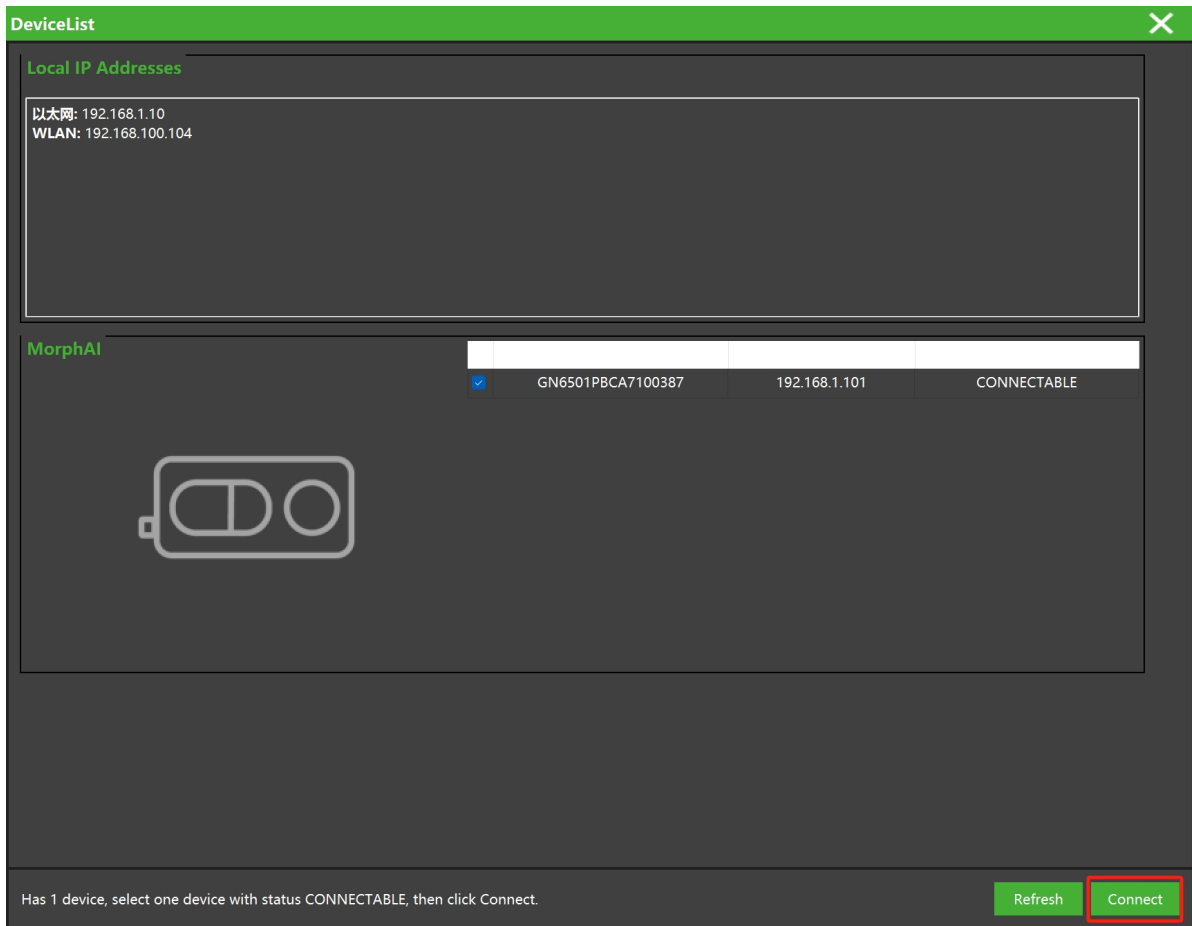
1. 搜索设备



2. 选中需要打开的设备

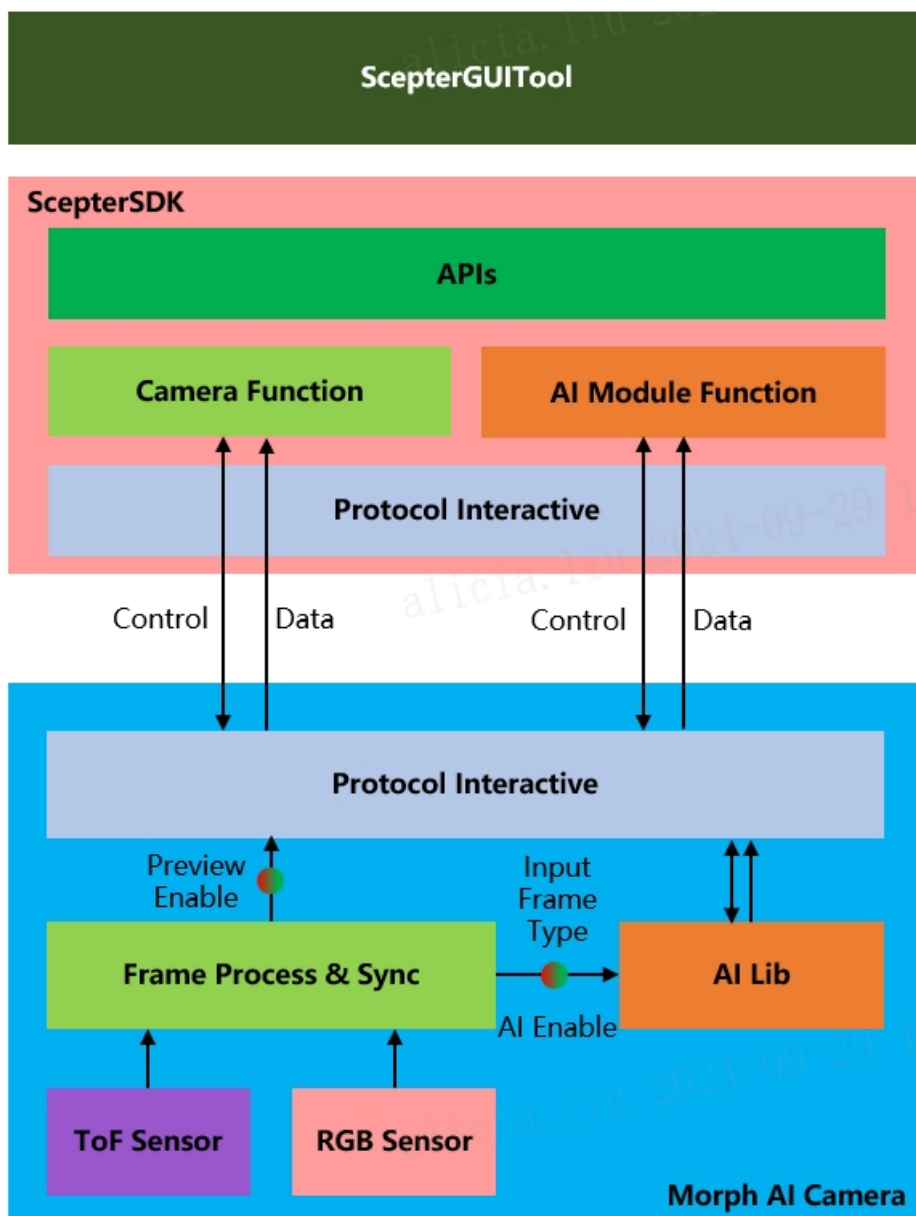


3. 点击Connect，打开设备



3 功能介绍

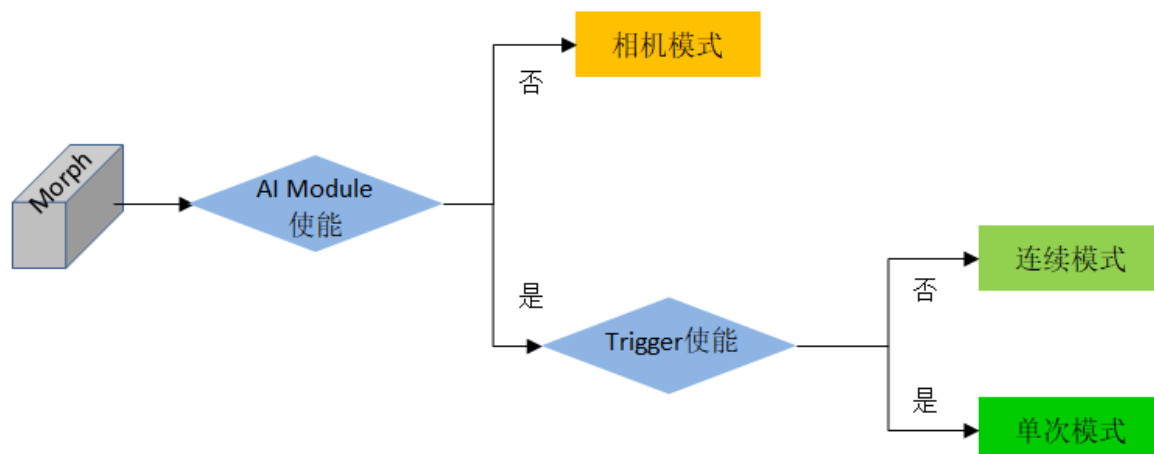
Morph AI平台的整体架构如下，与标准RGBD相机相比增加了AI-Lib模块，可以与ScepterSDK直接交互。ScepterGUITool 图形界面工具是基于 Scepter SDK 开发，并提供了部分调试功能。



Morph相机功能需要搭配ScepterSDK与ScepterGUITool使用，功能分为标准相机功能与算法功能两部分，并且可以通过开关，分别进行单独控制。

AI Module关闭，相机为标准相机模式，仅上传图像；

AI Module打开，通过InputFrameType选择输入AI Module的图像类型，经过算法模块计算后，可以在标准相机模式上传图像的基础上，上传算法结果。另外算法还支持单次、连续两种运行模式。



ScepterSDK以功能划分，将相机功能与算法功能相关的API分别包含在头文件Scepter_api.h与Scepter_Morph_api.h中。在Samples/Moprh中提供了多个示例，帮助开发者了解如何使用接口，开发自己的功能。

示例名称	说明
AIContinuousRunMode	连续运行模式的初始化、设置、算法数据获取方法
AISingleRunMode	单次运行模式的初始化、设置、触发方法、算法数据获取方法
AIGetFrameAndResultSync	图像数据获取，算法数据获取方法，以及如何进行同步判断，匹配同一时刻算法与图像

ScepterSDK目录：

ScepterSDK > BaseSDK > Windows > Include

名称	修改日期	类型
 Scepter_api.h	2024/10/12 10:16	C/C++ Header
 Scepter_define.h	2024/10/12 10:16	C/C++ Header
 Scepter_enums.h	2024/10/12 10:16	C/C++ Header
 Scepter_Morph_api.h	2024/10/12 10:16	C/C++ Header
 Scepter_types.h	2024/10/12 10:16	C/C++ Header

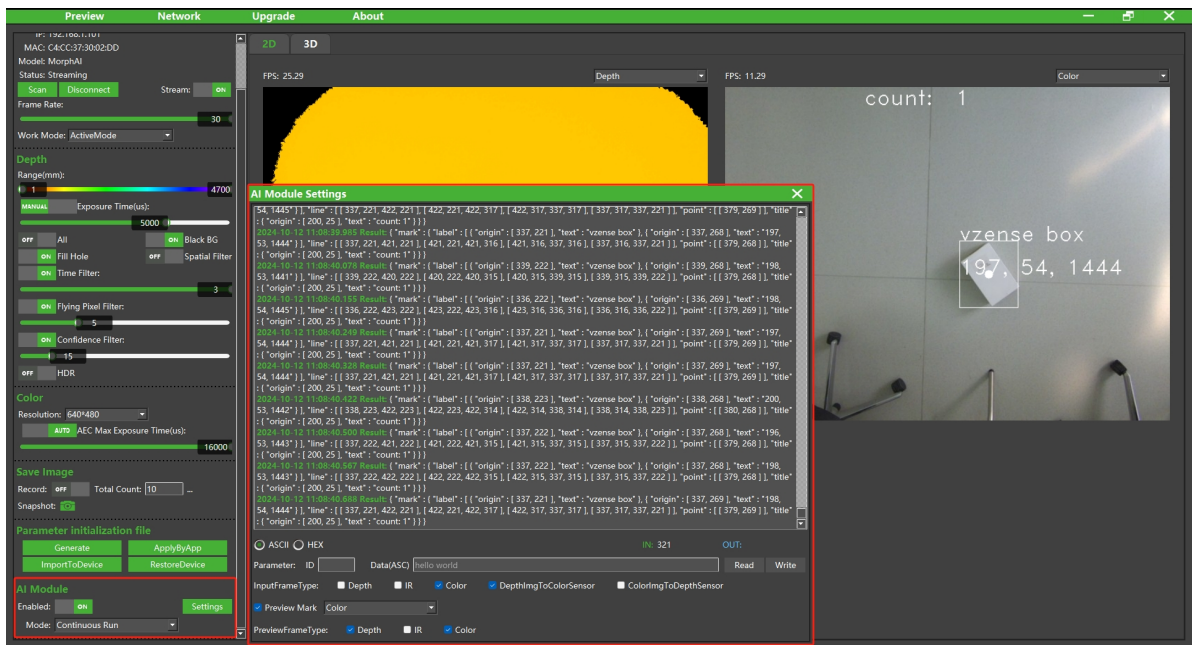
ScepterSDK > BaseSDK > Windows > Samples > Base > Morph >

名称	修改日期	类型	大小
 AIContinuousRunMode	2024/10/12 10:16	文件夹	
 AIGetFrameAndResultSync	2024/10/12 10:16	文件夹	
 AISingleRunMode	2024/10/12 10:16	文件夹	
 Morph_Samples.sln	2024/10/12 10:16	Visual Studio Soluti...	

ScepterGUITool

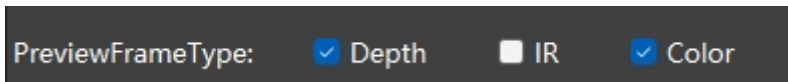
ScepterGUITool在打开Morph AI相机时，不仅包含与标准相机相同的所有图像设置相关功能，还会自动检测并增加AI Module控制模块界面。该控制模块可以控制算法模块的相关功能，如Module使能、工作模式设置、相关参数设置等。

ScepterGUITool的AI Module内容如下：



3.1 图像预览

如上图所示，设备连接成功后，可以在软件左侧，看到设备信息和控制选项。点击Stream选项的ON/OFF开关，开启相机数据流。



AI Module Settings子窗口最下方的PreviewFrameType用来控制相机图像预览类型，其中包含Depth，IR和Color三种。取消勾选某一种图像，则相机停止该类型图像的上传预览。

图像类型的预览开关，不影响算法实际输入，具体请参考第一章架构图。

SDK API:

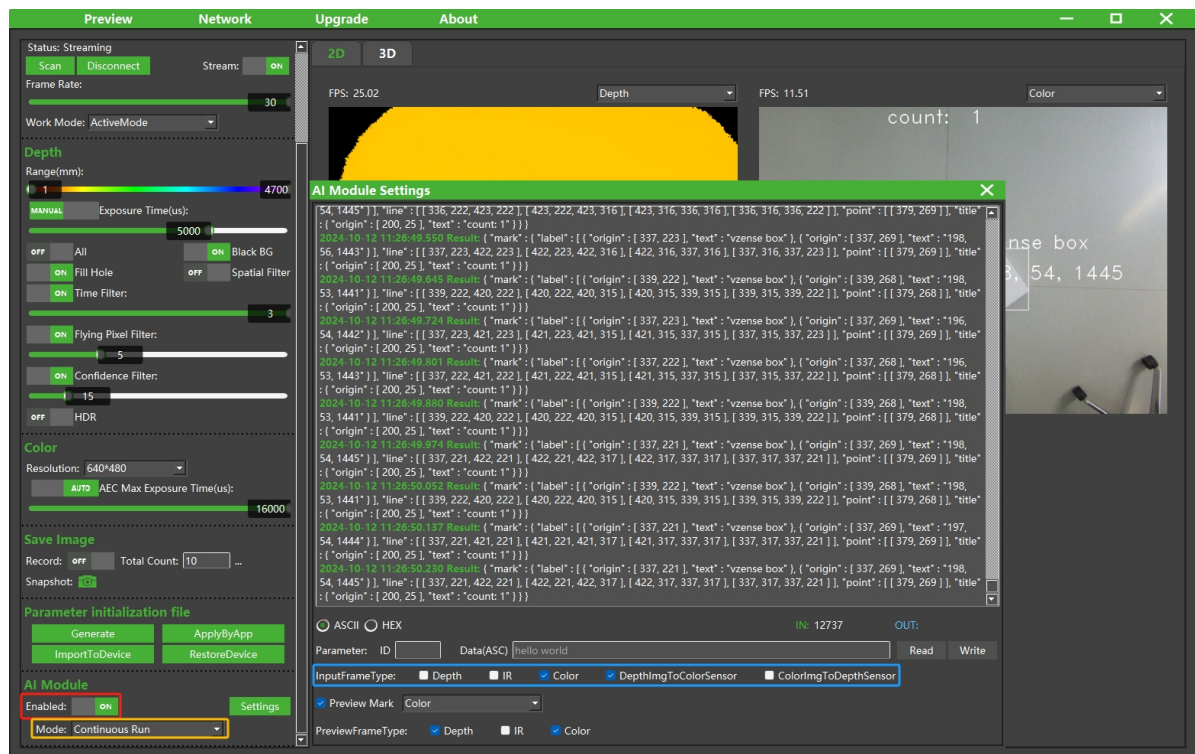
```
1 //开流
2 ScStatus scStartStream(ScDeviceHandle device)
3 //获取图像
4 ScStatus scGetFrameReady(ScDeviceHandle device, uint16_t waitTime,
5 ScFrameReady* pFrameReady)
6 ScStatus scGetFrame(ScDeviceHandle device, ScFrameType frameType, ScFrame*
7 pScFrame)
```

参考例程:

ScepterSDK\BaseSDK\windows\Samples\Base\NYX650\DeviceStartStopStreaming

3.2 AI Module控制

为了方便开发者快速进行计算机视觉项目的开发，Morph AI相机提供了AI Module控制功能，包含算法使能，工作模式及图像输入类型设置三部分。



- 模块使能

Morph AI相机运行时，默认不启动AI Module功能，此时相机与标准相机相同。点击AI Module的Enable开关，使能算法模块，此时相机开启算法相关功能。

点击Enable开关后，Enable开关自动调回关闭状态，则表明算法库调用失败。建议检查算法库是否工作异常。

- 工作模式

通过AI Module的Mode可以选择当前算法的工作模式，支持 Continuous Run 和 Single Run 两种工作模式。

- **Continuous Run**：连续运行模式下，算法对每张图片进行处理，输出算法结果。
- **Single Run**：单次运行模式下，算法只有接收到触发信号时才会进行一次图像处理，并输出图像对应的算法结果。

- 图像输入类型

通过InputFrameType勾选进行算法输入图像的选择，具体图像类型根据开发者需求而定。

SDK API:

```
1 //算法使能
2 ScStatus scaModuleSetEnabled(ScDeviceHandle device, bool bEnabled)
3 ScStatus scaModuleGetEnabled(ScDeviceHandle device, bool* pEnabled)
4
5 //算法工作模式设置
6 ScStatus scaModuleSetWorkMode(ScDeviceHandle device, SCAIModuleMode mode)
7 ScStatus scaModuleGetWorkMode(ScDeviceHandle device, SCAIModuleMode* mode)
8 ScStatus scaModuleTriggerOnce(ScDeviceHandle device)
9
10 //图像输入类型
```

```

11 ScStatus scAImoduleSetInputFrameTypeEnabled(ScDeviceHandle device,
12 ScFrameType frameType, bool bEnabled)
13 //参数中frameType表示待获取图像的类型
14 ScStatus scGetFrame(ScDeviceHandle device, ScFrameType frameType, ScFrame*
15 pScFrame)
16 //ScFrameType枚举值
17 typedef enum
18 {
19     SC_DEPTH_FRAME = 0,
20     SC_IR_FRAME = 1,
21     SC_COLOR_FRAME = 3,
22     SC_TRANSFORM_COLOR_IMG_TO_DEPTH_SENSOR_FRAME = 4,
23     SC_TRANSFORM_DEPTH_IMG_TO_COLOR_SENSOR_FRAME = 5,
24 } ScFrameType;

```

参考例程:

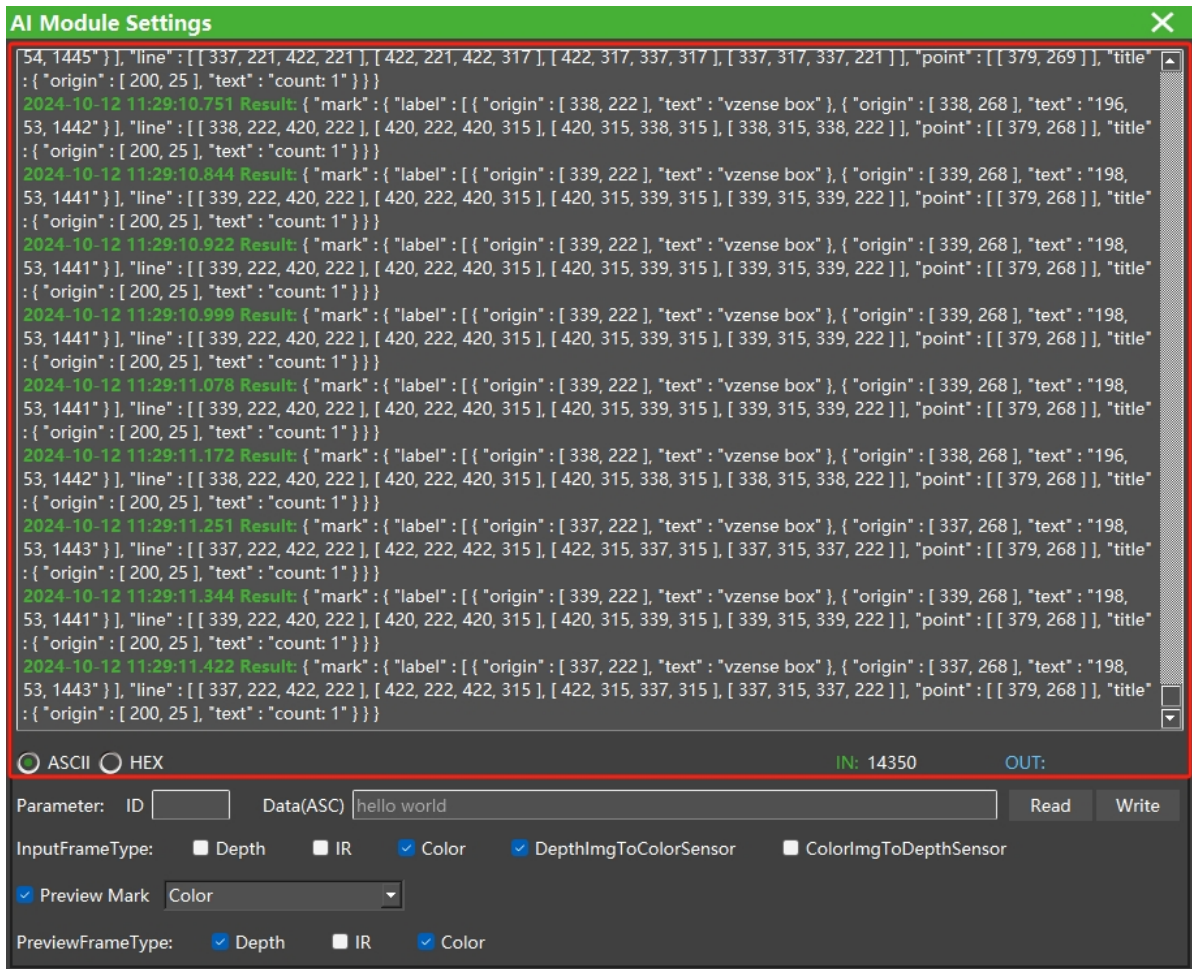
ScepterSDK\BaseSDK\windows\Samples\Base\Morph\AISingleRunMode

ScepterSDK\BaseSDK\windows\Samples\Base\Morph\AIContinuousRunMode

ScepterSDK\BaseSDK\windows\Samples\Base\Morph\AIGetFrameAndResultSync

3.3 结果输出

算法结果输出框位于AI Module Settings子界面最上方，支持ASCII和HEX两种显示格式，开发者可以根据需求自由切换。同时结果输出框右下角的IN，OUT会进行数据统计，其中IN表示工具接收到信息的条数（包含算法结果，算法info，算法参数等），OUT表示写入设备的信息条数。



SDK API:

```
1 //算法结果获取
2 ScStatus ScAIModuleGetResult(ScDeviceHandle device, uint16_t waitTime,
    ScAIResult* pAIResult)
```

参考例程:

ScepterSDK\BaseSDK\Windows\Samples\Base\Morph\AIContinuousRunMode

3.4 参数I/O

参数I/O功能提供设备算法模块与上位机应用交互信息的通道，方便开发者进行功能自定义实现，如算法参数设置与读取。参数I/O通过ID与Data进行参数的数据交换，详细介绍如下：



ID: 参数的类型ID输入框，数据类型uint32，数据范围[0x00000000: 0xFFFFFFFF]。

Data: 算法参数输入框，根据实际算法需求设置，支持ASCII和HEX。

Read: 读取按钮，读取相机中的AI模块的参数，并显示在结果输出界面与Data文本框。

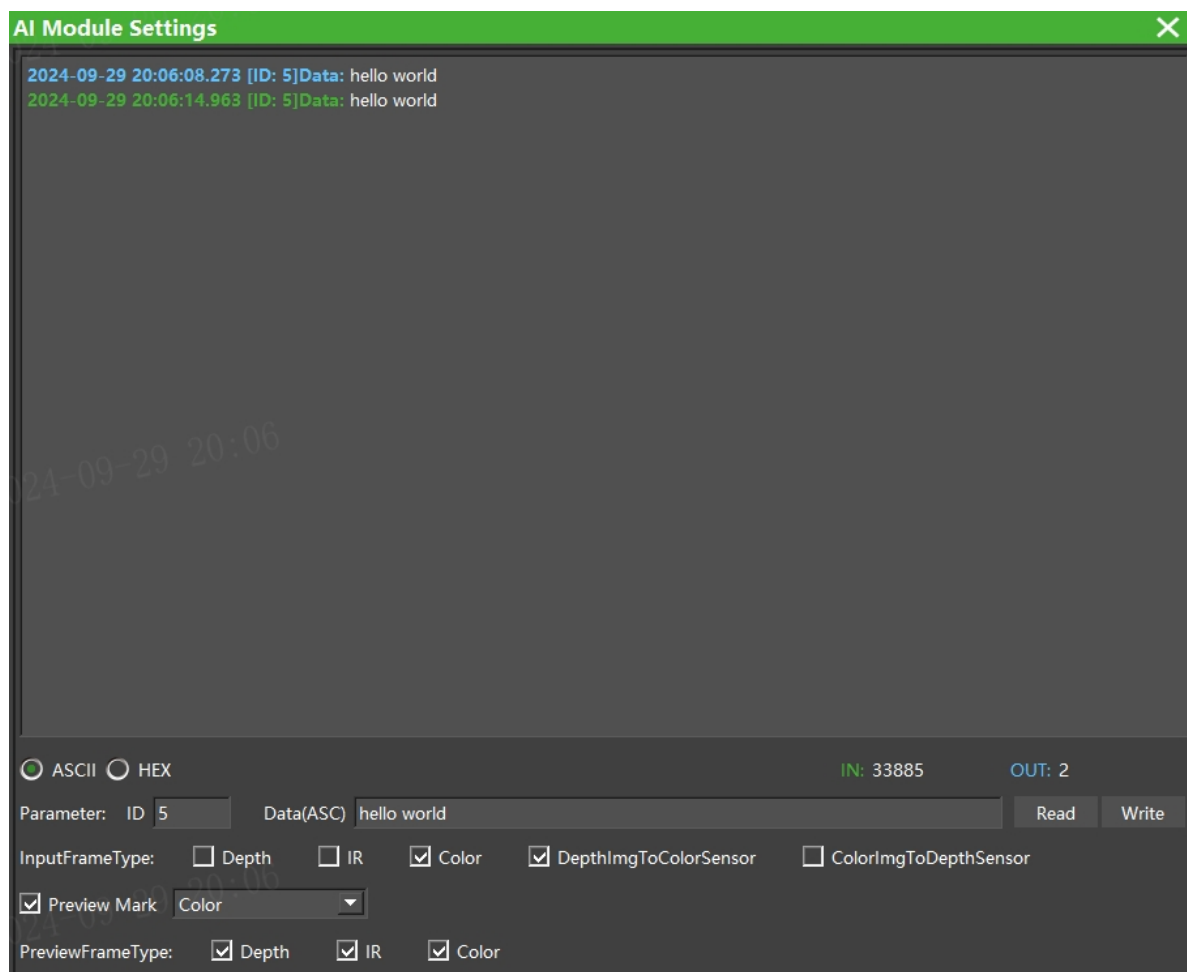
Write: 写入按钮，将填写在Data文本框中的内容写入相机，传递给算法模块。

下面以预置算法库进行测试举例：

参数写入： ID输入5，在文本框中写入ASCII字符“hello world”，点击Write，数据写入算法模块，如下图。

参数读取： ID输入5，删除文本框中内容，点击Read读取相机参数，如下图。

实际操作中，需注意ASCII与HEX的选择，应与预期与需求一致。



SDK API:

```
1 //算法模型参数设置
2 ScStatus scAIModuleSetParam(ScDeviceHandle device, uint32_t paramID, void*
  pBuffer, uint16_t bufferSize)
3 ScStatus scAIModuleGetParam(ScDeviceHandle device, uint32_t paramID, void**
  pBuffer, uint16_t *pBufferSize)
```

参考例程：

ScepterSDK\BaseSDK\windows\Samples\Base\Morph\AIContinuousRunMode

3.5 Mark功能

ScepterGUITool提供预览图Mark功能，该功能可以在预览图像上显示算法检测结果，帮助开发者直观验证算法输出。使用该功能需要按照如下方式进行：

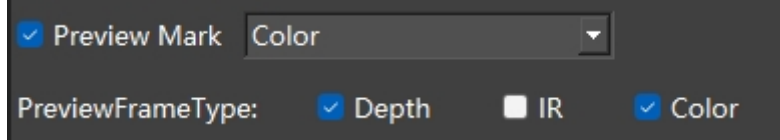
1. 使能Preview Mark功能，并选择合适的图像类型。即算法输出的mark符合规范时，图像上会显示算法标记信息。
2. 使能选择要进行Mark标记的预览图像
3. 算法输入结果使用GUITool可识别的格式内容进行结果输出。

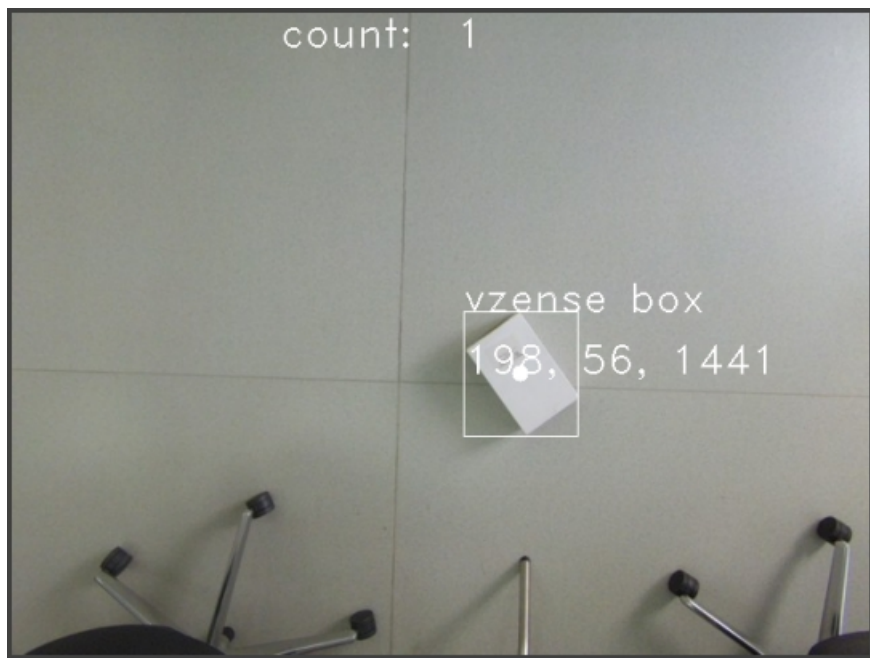
ScepterGUITool可识别的Mark格式为Json格式内容，具体可参考如下内容，支持类型为title， point， line与label。

- point， line， label格式可支持多个目标
- title暂时只支持唯一

```
1 //mark格式举例说明，其中title只有一条，其余label，line，point根据算法输出的目标个数而定
2 {
3     "mark": {
4         "label": [
5             {
6                 "origin": [337,223],
7                 "text": "vzense box"
8             },
9             {
10                "origin": [337,269],
11                "text": "198, 56, 1441"
12            }
13        ],
14        "line": [[337,223,422,223],
15                [422,223,422,316],
16                [422,316,337,316],
17                [337,316,337,223]],
18        "point": [[379,269]],
19        "title": {
20            "origin": [200,25],
21            "text": "count: 1"
22        }
23    }
24 }
```

```
2024-10-12 11:39:57.972 Result: { "mark": { "label": [ { "origin": [ 337, 223 ], "text": "vzense box" }, { "origin": [ 337, 269 ], "text": "198, 56, 1441" } ], "line": [ [ 337, 223, 422, 223 ], [ 422, 223, 422, 316 ], [ 422, 316, 337, 316 ], [ 337, 316, 337, 223 ] ], "point": [ [ 379, 269 ] ], "title": { "origin": [ 200, 25 ], "text": "count: 1" } } }
```





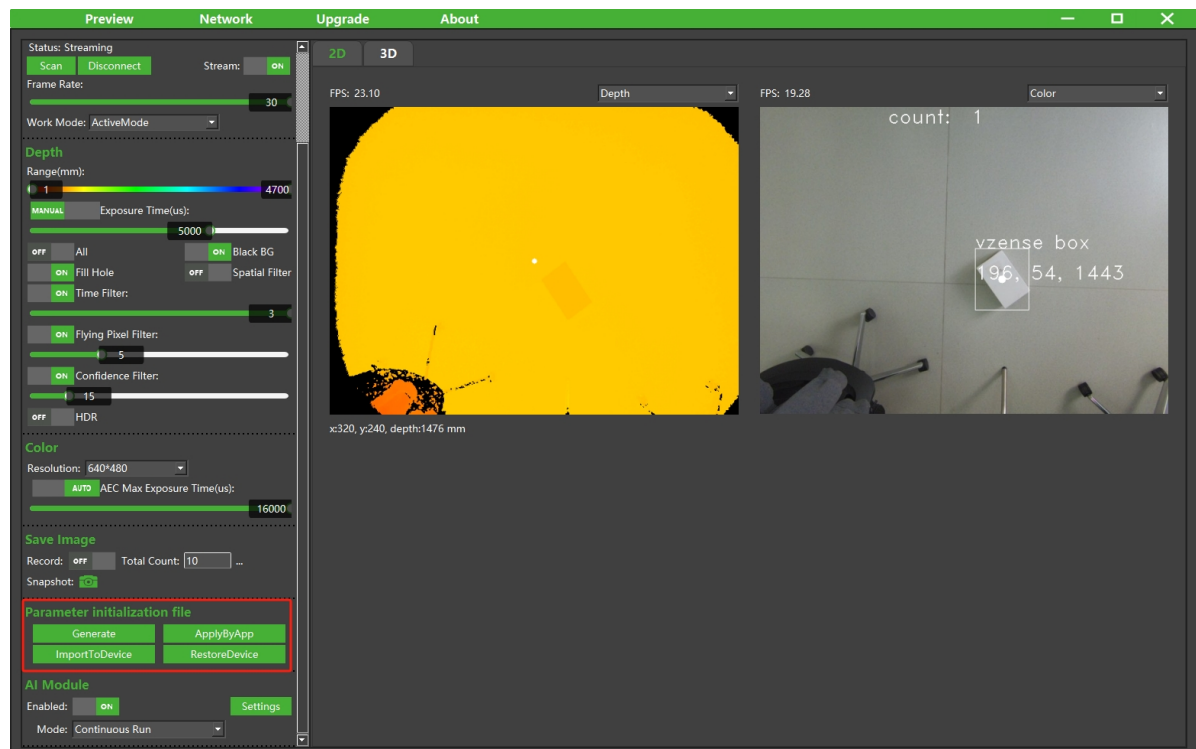
SDK API:

```
1 //获取算法结果
2 ScStatus ScAImoduleGetResult(ScDeviceHandle device, uint16_t waitTime,
    ScAImoduleResult* pAImoduleResult)
```

Mark功能为ScepterGUITool实现的功能，SDK只实现分别上传图像和算法结果的功能，不把算法结果绘制到图像上。

3.6 默认参数设置

Morph AI相机支持默认参数设置功能，帮助开发者快速部署相机与算法参数，使得相机上电初始化后直接以设置参数运行，省掉多余的接口调用。



使用方法:

- 点击 `Generate`，可以保存当前的参数到配置文件中，配置文件保存在JsonConfig文件夹下，以“相机品类_年_月_日_时_分_秒_SN.json”格式命名。
- 点击 `ImportToDevice`，选择配置文件，提示导入成功后，重启相机参数生效。
- 点击 `ApplyByApp`，选择配置文件，提示导入成功后，当次生效，重启相机后参数还是出厂设置的默认参数。
- 点击 `RestoreDevice`，相机参数重置为出厂默认设置。

相机相关参数与普通相机一致，AI算法相关参数包含算法使能，算法工作模式，输入图像类型，预览图像类型等。

AI算法相关参数如下：

```
1  //AI算法相关参数
2  "MorphAI": {
3      "AIModuleMode": "ContinuousRunMode",
4      "Enable": 1,
5      "InputFrameType": [
6          "Color",
7          "TransformedDepth"
8      ],
9      "PreviewFrameType": [
10         "Depth",
11         "Color"
12     ]
13 }
```

SDK API:

```
1  //SDK解析json配置文件，并把对应参数设置给相机，对应ScepterGUITool中按钮【ApplyByApp】
   的功能
2  ScStatus  scSetParamsByJson(ScDeviceHandle device, char* pfilePath)
3  //SDK把json配置文件直接下发给相机，对应ScepterGUITool中按钮【ImportToDevice】的功能
4  ScStatus  scImportParamInitFile(ScDeviceHandle device, char* pfilePath)
5  //重置相机参数，对应ScepterGUITool中按钮【RestoreDevice】的功能
6  ScStatus  scRestoreParamInitFile(ScDeviceHandle device)
```

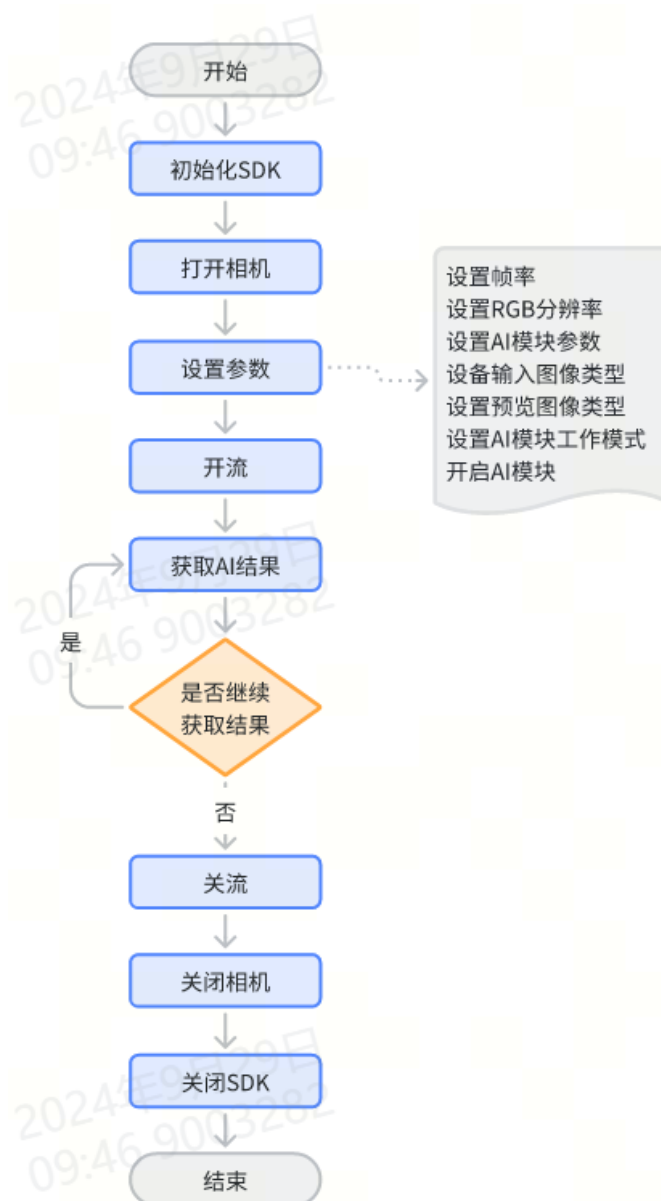
参考例程:

ScepterSDK\BaseSDK\windows\Samples\Base\NYX650\DeviceSetParamsByJson

4 SDK例程

为了帮助开发者更快的使用Morph AI相机和ScepterSDK开发自己的算法应用，此处以连续工作模式下AI算法开发为例，介绍应用的通用处理流程和相关代码。

AIContinuousRunMode例程流程图如下：



首先初始化SDK，获取当前设备数目和设备列表，打开目标相机后进行相机和算法参数设置，开流后将图像数据输入算法库，就可以获得算法结果，在程序结束前，要先关流，关闭相机，关闭SDK，之后完整例程结束。

参考例程时需要注意以下几点：

- 例程中相关参数是使用API进行的设置。在实际开发中，可以根据应用场景，通过ScepterGUITool调试好相关参数，然后导出配置文件，再把配置文件导入到使用的相机中。这样就无需使用API对相机进行参数设置，可以注释掉示例中的相关代码，减少函数调用，缩短应用启动时间。
- 使用scAIModuleSetInputFrameTypeEnabled设置算法的输入图像类型时，根据需求只设置所需图像类型即可，避免设置多余图像类型，造成算力浪费。
- 预览图像队列和算法结果队列都包含硬件时间戳。如果需要同步预览图像和算法结果，可以通过检测时间戳是否一致进行匹配，请参考AIGetFrameAndResultSync例程。
- 图像预览功能如果在生产环境中不需要，可以通过接口或者配置文件关闭，这样可以节省网络带宽和设备算力。

示例代码

ScepterSDK\BaseSDK\Windows\Samples\Base\Morph\AIContinuousRunMode\AIContinuousRunMode.cpp如下，其他例程请参考对应代码。

```

1  #include <thread>
2  #include <iostream>
3  #include <iomanip>
4  #include "Scepter_api.h"
5  #include "Scepter_Morph_api.h"
6
7  #define ResultCount 100
8  using namespace std;
9
10 int SetParams(ScDeviceHandle deviceHandle)
11 {
12     ScStatus status = SC_OTHERS;
13     //Set frame rate
14     status = scSetFrameRate(deviceHandle, 15);
15     if (status != ScStatus::SC_OK)
16     {
17         cout << "scSetFrameRate failed status:" << status << endl;
18         return -1;
19     }
20     //Set color resolution
21     status = scSetColorResolution(deviceHandle, 640, 480);
22     if (status != ScStatus::SC_OK)
23     {
24         cout << "scSetColorResolution failed status:" << status << endl;
25         return -1;
26     }
27
28     //Get and set paramID 4
29     uint32_t paramID = 4;
30     void* pBuffer = 0;
31     uint16_t nBufferSize = 0;
32     status = sCAIModuleGetParam(deviceHandle, paramID, &pBuffer,
33     &nBufferSize);
34     if (status != ScStatus::SC_OK)
35     {
36         cout << "sCAIModuleGetParam paramID: 4 failed status:" << status <<
37         endl;
38         return -1;
39     }
40     string strBuffer = (char*)pBuffer;
41     cout << "sCAIModuleGetParam paramID status:" << status << " text: "<<
42     strBuffer.c_str() <<endl;
43
44     char buffer_ASCII[13] = {"Hello world."};
45     status = sCAIModuleSetParam(deviceHandle, paramID, buffer_ASCII,
46     sizeof(buffer_ASCII));
47     if (status != ScStatus::SC_OK)
48     {
49         cout << "sCAIModuleSetParam paramID: 4 failed status:" << status <<
50         endl;
51         return -1;
52     }
53     cout << "sCAIModuleSetParam paramID status:" << status << " text: " <<
54     buffer_ASCII << endl;
55     status = sCAIModuleGetParam(deviceHandle, paramID, &pBuffer,
56     &nBufferSize);
57     if (status != ScStatus::SC_OK)
58     {

```

```

52     cout << "SCAIModuleGetParam paramID: 4 failed status:" << status <<
endl;
53     return -1;
54 }
55 strBuffer = (char*)pBuffer;
56 cout << "SCAIModuleGetParam paramID status:" << status << "    text: "
<< strBuffer.c_str() << endl;
57
58 //Get and set paramID 5
59 paramID = 5;
60 status = SCAIModuleGetParam(deviceHandle, paramID, &pBuffer,
&nBufferSize);
61 if (status != ScStatus::SC_OK || nBufferSize < 1)
62 {
63     cout << "SCAIModuleGetParam paramID: 5 failed status:" << status <<
endl;
64     return -1;
65 }
66
67 uint8_t* pbuf = (uint8_t*)pBuffer;
68 cout << "SCAIModuleGetParam paramID: 5 status:" << status << "    text:
" << hex;
69 for (int i = 0; i < nBufferSize; i++)
70 {
71     cout << "0x" << setfill('0') << std::setw(2) << static_cast<int>(*
(pbuf + i)) << " ";
72 }
73 cout << dec << endl;
74
75 uint8_t buffer_HEX[3] = { 0x01, 0x03, 0x03 };
76 cout << "SCAIModuleSetParam paramID: 5 status:" << status << "    text:
";
77 for (int i = 0; i < sizeof(buffer_HEX); i++)
78 {
79     cout << "0x" << setfill('0') << std::setw(2) << hex <<
static_cast<int>(buffer_HEX[i]) << " ";
80 }
81 cout << dec << endl;
82 status = SCAIModuleSetParam(deviceHandle, paramID, buffer_HEX,
sizeof(buffer_HEX));
83 if (status != ScStatus::SC_OK)
84 {
85     cout << "SCAIModuleSetParam paramID: 5 failed status:" << status <<
endl;
86     return -1;
87 }
88
89 paramID = 5;
90 status = SCAIModuleGetParam(deviceHandle, paramID, &pBuffer,
&nBufferSize);
91 if (status != ScStatus::SC_OK || nBufferSize < 1)
92 {
93     cout << "SCAIModuleGetParam paramID: 5 failed status:" << status <<
endl;
94     return -1;
95 }
96
97 pbuf = (uint8_t*)pBuffer;

```

```

98     cout << "sCAIModuleGetParam paramID: 5 status:" << status << "    text:"
    << hex;
99     for (int i = 0; i < nBufferSize; i++)
100     {
101         cout << "0x" << setfill('0') << std::setw(2) << static_cast<int>(*
(pbuf + i)) << " ";
102     }
103     cout << dec << endl;
104
105     //Set input frame type color
106     status = sCAIModuleSetInputFrameTypeEnabled(deviceHandle,
SC_COLOR_FRAME, true);
107     if (status != ScStatus::SC_OK)
108     {
109         cout << "sCAIModuleSetInputFrameTypeEnabled color failed status:"
<< status << endl;
110         return -1;
111     }
112
113     //Set input frame type transfomedDepth
114     status = sCAIModuleSetInputFrameTypeEnabled(deviceHandle,
SC_TRANSFORM_DEPTH_IMG_TO_COLOR_SENSOR_FRAME, true);
115     if (status != ScStatus::SC_OK)
116     {
117         cout << "sCAIModuleSetInputFrameTypeEnabled transfomedDepth failed
status:" << status << endl;
118         return -1;
119     }
120
121     //Set preview frame type depth false
122     status = sCAIModuleSetPreviewFrameTypeEnabled(deviceHandle,
SC_DEPTH_FRAME, false);
123     if (status != ScStatus::SC_OK)
124     {
125         cout << "sCAIModuleSetPreviewFrameTypeEnabled depth failed status:"
<< status << endl;
126         return -1;
127     }
128
129     //Set preview frame type IR false
130     status = sCAIModuleSetPreviewFrameTypeEnabled(deviceHandle,
SC_IR_FRAME, false);
131     if (status != ScStatus::SC_OK)
132     {
133         cout << "sCAIModuleSetPreviewFrameTypeEnabled IR failed status:" <<
status << endl;
134         return -1;
135     }
136
137     //Set AI enable work mode
138     status = sCAIModuleSetWorkMode(deviceHandle, AI_CONTINUOUS_RUN_MODE);
139     if (status != ScStatus::SC_OK)
140     {
141         cout << "sCAIModuleSetWorkMode failed status:" << status << endl;
142         return -1;
143     }
144
145     //Set AI enable

```

```

146     status = SCAModuleSetEnabled(deviceHandle, true);
147     if (status != ScStatus::SC_OK)
148     {
149         cout << "SCAModuleSetEnabled failed status:" << status << endl;
150         return -1;
151     }
152
153     return 0;
154 }
155
156 int main()
157 {
158     cout << "---AIContinuousRunMode---" << endl;
159
160     uint32_t deviceCount;
161     ScDeviceInfo* pDeviceListInfo = NULL;
162     ScDeviceHandle deviceHandle = 0;
163     ScStatus status = SC_OTHERS;
164
165     //SDK initialize
166     status = scInitialize();
167     if (status != ScStatus::SC_OK)
168     {
169         cout << "scInitialize failed status:" << status << endl;
170         system("pause");
171         return -1;
172     }
173
174     //1.Search and notice the count of devices.
175     //2.get infomation of the devices.
176     //3.open devices accroding to the info.
177     status = scGetDeviceCount(&deviceCount, 3000);
178     if (status != ScStatus::SC_OK)
179     {
180         cout << "scGetDeviceCount failed! make sure pointer valid or called
181         scInitialize()" << endl;
182         system("pause");
183         return -1;
184     }
185     cout << "Get device count: " << deviceCount << endl;
186     if (0 == deviceCount)
187     {
188         cout << "scGetDeviceCount scans for 3000ms and then returns the
189         device count is 0. Make sure the device is on the network before running
190         the samples."<< endl;
191         system("pause");
192         return -1;
193     }
194
195     pDeviceListInfo = new ScDeviceInfo[deviceCount];
196     status = scGetDeviceInfoList(deviceCount, pDeviceListInfo);
197     if (status == ScStatus::SC_OK)
198     {
199         if (SC_CONNECTABLE != pDeviceListInfo[0].status)
200         {
201             cout << "connect status: " << pDeviceListInfo[0].status <<
202             endl;
203             cout << "The device state does not support connection."<< endl;

```



```

200         return -1;
201     }
202 }
203 else
204 {
205     cout << "GetDeviceListInfo failed status:" << status << endl;
206     return -1;
207 }
208
209 cout << "serialNumber:" << pDeviceListInfo[0].serialNumber << endl
210 << "ip:" << pDeviceListInfo[0].ip << endl
211 << "connectStatus:" << pDeviceListInfo[0].status << endl;
212
213 status = scOpenDeviceBySN(pDeviceListInfo[0].serialNumber,
&deviceHandle);
214 if (status != SCStatus::SC_OK)
215 {
216     cout << "OpenDevice failed status:" << status << endl;
217     return false;
218 }
219
220 //if the setting param come from json file or ScepterGUITool, the
following interfaces can be annotation.
221 if (0 != SetParams(deviceHandle))
222 {
223     return -1;
224 }
225
226 //Starts capturing the image stream
227 status = scStartStream(deviceHandle);
228 if (status != SCStatus::SC_OK)
229 {
230     cout << "scStartStream failed status:" << status << endl;
231     return -1;
232 }
233
234 //Get result
235 for (int j = 0; j < ResultCount; j++)
236 {
237     SCAIResult aiResult = { 0 };
238     SCStatus status = scaIModuleGetResult(deviceHandle, 1200,
&aiResult);
239     if (status != SC_OK)
240     {
241         cout << " scaIModuleGetResult failed status:" << status <<
endl;
242         std::this_thread::sleep_for(std::chrono::milliseconds(5));
243         continue;
244     }
245     cout << "AIModuleGetResult resultIndex :" << aiResult.resultIndex
<< endl;
246 }
247
248 //Stop capturing the image stream
249 status = scStopStream(deviceHandle);
250 if (status != SCStatus::SC_OK)
251 {
252     cout << "scStopStream failed status:" << status << endl;

```

```
253         return -1;
254     }
255
256     //1.close device
257     //2.SDK shutdown
258     status = scCloseDevice(&deviceHandle);
259     if (status != ScStatus::SC_OK)
260     {
261         cout << "scCloseDevice failed status:" << status << endl;
262         return -1;
263     }
264     status = scShutdown();
265     if (status != ScStatus::SC_OK)
266     {
267         cout << "scShutdown failed status:" << status << endl;
268         return -1;
269     }
270     cout << "---end---" << endl;
271
272     return 0;
273 }
274
```